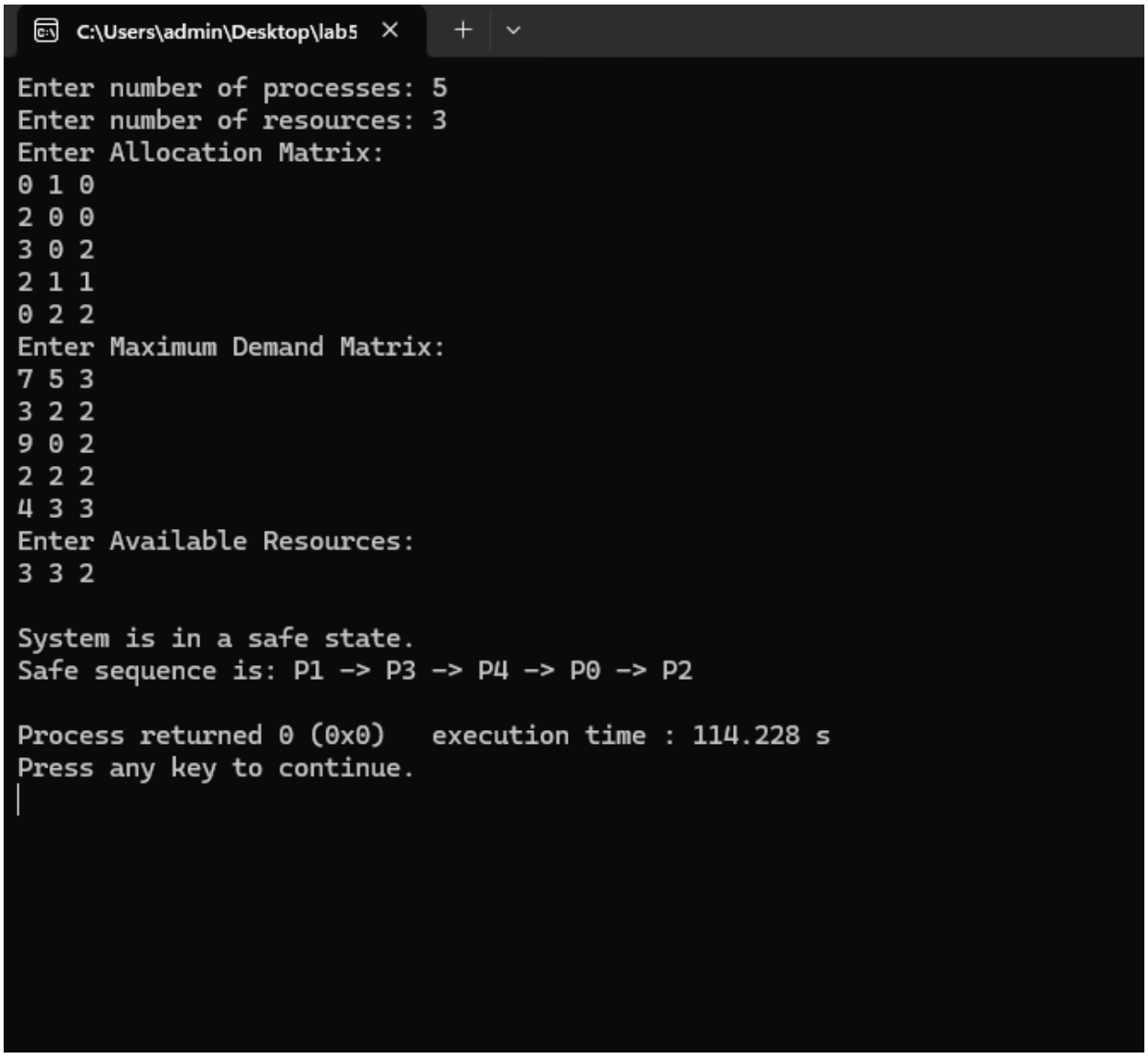
Write a C program to simulate
a. Bankers' algorithm for the purpose of deadlock avoidance
Write safety algorithm, program and output

```
#include <stdio.h>

int main()
{
   int n, m, i, j, k;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter number of resources: ");
   scanf("%d", &m);

   int alloc[n][m], max[n][m], need[n][m];
   int avail[m];

   printf("Enter Allocation Matrix:\n");
   for(i = 0; i < n; i++)
   {
      for(j = 0; j < m; j++)
      {
         scanf("%d", &alloc[i][j]);
      }
   }

   printf("Enter Maximum Demand Matrix:\n");
   for(i = 0; i < n; i++)
   {
      for(j = 0; j < m; j++)
      {
         scanf("%d", &max[i][j]);
      }
```

```
}

printf("Enter Available Resources:\n");
for(i = 0; i < m; i++)
{
    scanf("%d", &avail[i]);
}

for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

int finish[n], safeSeq[n];
int work[m];

for(i = 0; i < m; i++)
    work[i] = avail[i];

for(i = 0; i < n; i++)
    finish[i] = 0;

int count = 0;

while(count < n)
{
    int found = 0;

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            for(j = 0; j < m; j++)
            {
```

```
            if(need[i][j] > work[j])
                break;
        }

        if(j == m)
        {
            for(k = 0; k < m; k++)
            {
                work[k] += alloc[i][k];
            }

            safeSeq[count] = i;
            count++;
            finish[i] = 1;
            found = 1;
        }
    }
  }

  if(found == 0)
  {
    printf("\nSystem is NOT in safe state.\n");
    return 0;
  }
}

printf("\nSystem is in a safe state.\n");
printf("Safe sequence is: ");

for(i = 0; i < n; i++)
{
  printf("P%d", safeSeq[i]);

  if(i != n - 1)
    printf(" -> ");
}
```

```
    printf("\n");

    return 0;
}
```



b. Deadlock Detection
Write (several instance Bankers Algorithm) algorithm, program
and output

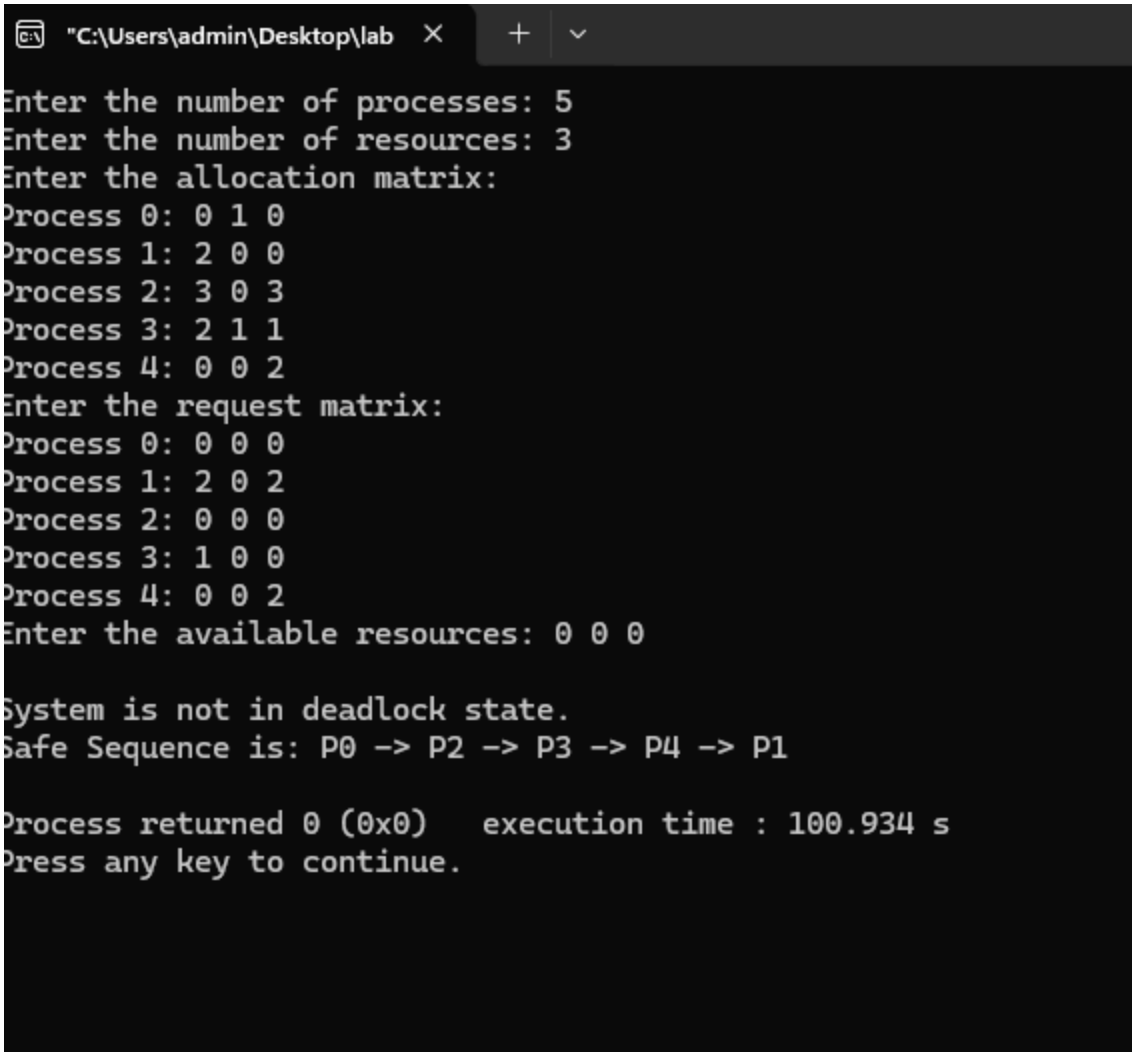
```
#include <stdio.h>
```

```
int main()
{
    int n, m, i, j, k;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    printf("Enter the number of resources: ");
    scanf("%d", &m);

    int allocation[n][m], request[n][m];
    int available[m];

    printf("Enter the allocation matrix:\n");
    for(i = 0; i < n; i++)
    {
        printf("Process %d: ", i);
        for(j = 0; j < m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("Enter the request matrix:\n");
    for(i = 0; i < n; i++)
    {
        printf("Process %d: ", i);
        for(j = 0; j < m; j++)
        {
            scanf("%d", &request[i][j]);
        }
    }

    printf("Enter the available resources: ");
    for(i = 0; i < m; i++)
    {
```

```
    scanf("%d", &available[i]);
}

int finish[n], safeSeq[n];
int work[m];

    for(i = 0; i < m; i++)
{
    work[i] = available[i];
}

    for(i = 0; i < n; i++)
{
    int zero = 1;

    for(j = 0; j < m; j++)
    {
        if(allocation[i][j] != 0)
        {
            zero = 0;
            break;
        }
    }

    if(zero)
        finish[i] = 1;
    else
        finish[i] = 0;
}

int count = 0;

while(count < n)
{
    int found = 0;

    for(i = 0; i < n; i++)
```

```
    {
        if(finish[i] == 0)
        {
            for(j = 0; j < m; j++)
            {
                if(request[i][j] > work[j])
                    break;
            }

            if(j == m)
            {
                for(k = 0; k < m; k++)
                {
                    work[k] += allocation[i][k];
                }

                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0)
        break;
}

int deadlock = 0;

for(i = 0; i < n; i++)
{
    if(finish[i] == 0)
    {
        deadlock = 1;
        printf("\nProcess P%d is deadlocked\n", i);
    }
}
```

```
    if(deadlock == 0)
    {
        printf("\nSystem is not in deadlock state.\n");
        printf("Safe Sequence is: ");

        for(i = 0; i < count; i++)
        {
            printf("P%d", safeSeq[i]);

            if(i != count - 1)
                printf(" -> ");
        }

        printf("\n");
    }

    return 0;
}
```


"C:\Users\admin\Desktop\lab
Enter the number of processes: 5
Enter the number of resources: 3
Enter the allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter the request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter the available resources: 0 0 0
System is not in deadlock state.
Safe Sequence is: P0 -> P2 -> P3 -> P4 -> P1
Process returned 0 (0x0) execution time : 100.934 s
Press any key to continue.